PAYMENT MANAGEMENT MODULE

Customer Order Management System | MySQL 8.0+

Project Documentation

This module extends the existing CustomerOrderDB with payment transaction tracking, status management, and revenue analysis.

1. Objectives

• Create a Payment table linked to the existing Orders table.

• Enforce one payment record at most for each order.

• Store payment modes and statuses using CHECK constraints.

• Manage payment transactions and retry status changes.

• Generate customer payment history and revenue reports.

2. Payment Table Design

Column	Data type	Constraint / purpose
Payment_ID	INT	PRIMARY KEY
Order_ID	INT	NOT NULL, UNIQUE, FOREIGN KEY → Orders(Order_ID)
Payment_Date	DATE	NOT NULL, defaults to current date
Payment_Mode	VARCHAR(30)	UPI / Credit Card / Debit Card / Net Banking / Cash on Delivery
Payment_Status	VARCHAR(20)	Pending / Success / Failed
Transaction_Amount	DECIMAL(12,2)	NOT NULL, CHECK >= 0

3. Updated ER Diagram

```
CUSTOMER (Customer_ID PK)
       1
       |
       | has
       | many
    ORDERS (Order_ID PK, Customer_ID FK)
       1
       |
       | has zero or one payment
       | (Payment.Order_ID is UNIQUE and FK)
       0..1
    PAYMENT (Payment_ID PK, Order_ID FK + UNIQUE)
PRODUCT (Product_ID PK) 1 ---- many ORDER_DETAILS many ---- 1 ORDERS
```

Relationship rule: each Payment row must reference an existing Orders row. Because Payment.Order_ID is UNIQUE, an order can have at most one payment record; an order can also have no payment record.

4. Sample Customer, Order and Payment Data

The module expects the existing Customer, Orders, Product, and Order_Details tables from customer_order_management.sql. The payment script inserts sample records for orders 1001, 1002, 1003, 1004, 1006, and 1007. Order 1005 is excluded because the base script deletes that cancelled order.

Payment_ID	Order_ID	Payment Mode	Status	Amount
501	1001	UPI	Success	1200.00
502	1002	Credit Card	Failed	2850.00
503	1003	Debit Card	Pending	1200.00
504	1004	Net Banking	Success	1800.00
506	1006	Cash on Delivery	Pending	2400.00
507	1007	UPI	Success	1950.00

5. Transaction Operations

• Insert new payment records for orders that do not already have a payment.

• Display all payments, successful payments, and failed or pending transactions.

• Update a failed or pending transaction to Success after a successful retry.

• Retrieve payment information for a specific Order_ID.

• Count successful and failed transactions and display full customer/order/payment details.

• Optional commented examples show how to mark a payment Failed or delete a payment row.

6. Payment Analysis and Revenue Reports

• Transaction count by payment mode and the most frequently used mode.

• Total successful payment revenue and successful revenue by payment mode.

• Average transaction amount.

• Customer payment history.

• Successful, failed, and pending transaction totals.

• Orders with failed or pending payments.

Revenue calculations include only rows whose Payment_Status is Success. The most-preferred-method report counts all payment rows regardless of status.

7. Execution Instructions

1. Open MySQL Workbench and connect to the server.

2. Run customer_order_management.sql first. It creates CustomerOrderDB and its existing tables.

3. Open and run payment_management.sql.

4. Review each SELECT result grid and confirm that the final duplicate-payment verification query returns zero rows.

Important: customer_order_management.sql includes DROP DATABASE IF EXISTS CustomerOrderDB. It will delete and recreate that database when executed. Back up important data before running it. The payment add-on does not drop the database.

8. Query Output Screenshots Checklist

■ Payment table rows after sample INSERT statements.

■ Successful payments query result.

■ Failed and pending transactions query result.

■ Complete payment details joined with customer and order information.

■ Transaction count by payment mode.

■ Successful payment revenue and revenue by payment method.

■ Customer payment history and orders with failed/pending payments.

Actual query-output screenshots are not embedded in this document. Capture them after executing the SQL script in MySQL Workbench and add them to the final submission.

9. Conclusion

The Payment Management Module demonstrates relational database design, primary and foreign keys, uniqueness and check constraints, payment status updates, joins, aggregation, and revenue reporting while linking payments to the existing Order Management System.